คำอธิบายการเขียนโปรแกรมจำลอง PFR
โดย Shooting Method (Backward Euler Newton)

ศุภวิชญ์ ศรีภูสิตโต

29 กันยายน 2025

แนวคิดของ Shooting Method

เริ่มจากการพิจารณาสมการเชิงอนุพันธ์พื้นฐานที่มีเงื่อนไขขอบเขต $y' = f(x, y),\ y(x_t) = \beta$ ในที่นี้เรานิยามให้ $y_t = y(x_t)$ เพื่อความสะดวกในการอธิบายภายหลัง สิ่งที่เป็นปัญหาคือที่ผ่านมาเราเจอแต่ ODE-IVP ที่กำหนดเงื่อนไขเริ่มต้นมาให้ แต่ตอนนี้เราเจอ ODE-BVP ที่ต้องเดาเงื่อนไขเริ่มต้นเอง หากเรานิยามค่าเดาเงื่อนไขเริ่มต้นคือ y_0 และเงื่อนไขเริ่มต้นที่แท้จริงคือ $\tilde{y_0}$ จะได้ลักษณะกราฟระหว่าง $x,\ y$ ดังรูป

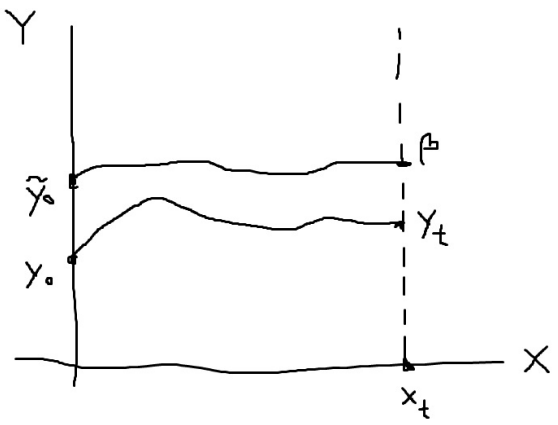


Figure 1: กราฟที่วาดด้วย paint.jpg

เราสรุปได้ว่ามีความสัมพันธ์บางอย่างที่โยง y_0 ไปสู่ y_t ใด ๆ (ที่จริงก็คือตัว ODE-IVP เองนั่นแหละ แค่เปลี่ยนรูปเล็กน้อย) เราจึงนิยามความสัมพันธ์นั้นคือ $y_t = \phi(y_0)$ หรือ $\beta = \phi(\tilde{y_0})$ หากจัดรูปใหม่ก็จะเป็น

$$\beta - \phi(\tilde{y_0}) = 0 \tag{1}$$

ซึ่งเราสามารถใช้ Newton's Method (หรือ Secant Method ที่เอามาทดแทน) เมื่อใช้ Newton's Method แล้ว จะได้ว่า

$$\tilde{y_0}^{i+1} = \tilde{y_0}^{i} + \frac{\beta - \phi}{\phi'}$$

หรือเมื่อจัดรูปเพื่อให้คิดขั้นตอนวิธีการคำนวณได้ง่ายขึ้น (เขียนโปรแกรมง่ายขึ้น & ปรับแก้ง่ายขึ้น) จะใช้ Secant Method ทำให้ได้สมการหลักที่จะนำไปใช้คือ

$$\tilde{y_0}^{i+1} = \tilde{y_0}^{i} + \varepsilon \frac{\beta - \phi}{\phi(\tilde{y_0}^{i+1} + \varepsilon) - y_t^{i+1}} \tag{2}$$

เมื่อ ε คือตัวแปรที่มีค่าน้อย ๆ (ส่วนตัวเราชอบใช้เลข 1×10^{-7} หรือ 1.06×10^{-8} เพราะค่ามันมากกว่า square root ของ machine epsilon ของคอมเราเล็กน้อย ถ้าคอมใครแม่นยำกว่าก็ใช้น้อยกว่านี้ได้ หรือคอมใครรุ่นเก่าก็อาจปรับค่าให้มากขึ้นได้บ้าง แค่ระวังว่าถ้ามากเกินอาจส่งผลต่อความเร็วในการคำนวณหรือความแม่นยำได้) ดังนั้นเราจึงแก้ ODEs-BVP ได้ โดยมีขั้นตอนคร่าว ๆ ดังนี้

1. เดาค่าเริ่มต้นของระบบ
2. หาคำตอบของระบบ
3. หาความคลาดเคลื่อนของตัวแปรต้น แล้วคำนวณตัวแปรต้นใหม่
4. วนกลับไปข้อที่ 2 จนกว่าความคลาดเคลื่อนจะน้อยพอใจเรา

โจทย์หลัก

ในการออกแบบ PFR เรามีระบบสมการดังนี้ (เราคิดมาให้แล้ว ไม่ต้องไปแทนค่าเอง)

$$k = 0.01exp\left(\frac{4800}{8.3145}\left(1/300 - 1/T\right)\right)$$

$$K_C = 10exp\left(\frac{-2200}{8.3145}(1/T - 1/450)\right)$$

$$-r_A = k\left(0.01(1-X_A)^2 - 0.1X_A/K_C\right)$$

$$dX_A/dV = 4(-r_A) \tag{3}$$

$$dT_a/dV = 0.384(T_a - T) \tag{4}$$

$$dT/dV = \frac{96(T_a - T) + 2200(-r_A)}{36} \tag{5}$$

โดยมีค่าขอบเขตคือ $X_A(0) = 0, T_a(10) = 450$ และ $T(0) = 300$ โดยสมการที่เป็นหัวใจหลักของแบบจำลองนี้คือ Energy Balance และ Design Equation ที่ระบุไว้ 3 สมการสุดท้าย เพื่อให้คำนวณได้ง่ายขึ้นเรานิยาม $\underline{y} = [X_A \ T_a \ T]^\top$ จะได้ว่า $\underline{y}' = \underline{f}(\underline{y})$ ขั้นแรกของการคำนวณเราเริ่มจากการเดาค่าอุณหภูมิโดยรอบของ reactor โดยเรารู้ว่าปฏิกิริยานี้คายพลังงาน ดังนั้นอุณหภูมิสุดท้ายควรมากกว่าอุณหภูมิเริ่มต้นไม่ว่าจะถ่ายโอนความร้อนรูปแบบใด ดังนั้นเราเดา $T_a(0) = 440$ ต่อมาเพื่อแก้ระบบ 3×3 โดยเราเลือกใช้ Implicit Euler (เพราะมันไม่เคยมีปัญหากับ ODE stiffness) รูปแบบสมการที่ได้จากการ discretize จึงเป็น

$$\underline{y}^{i+1} - \underline{y}^i - h\underline{f}(\underline{y}^{i+1}) = 0 \tag{6}$$

ซึ่งเราสามารถใช้ Newton's Method ในการแก้ปัญหาระบบพีชคณิตไม่เชิงเส้นนี้ได้ แต่สำหรับคอมพิวเตอร์ที่มันไม่สามารถหา gradient ของฟังก์ชันด้วยตัวเองเป็นจึงเป็นเรื่องยุ่งยากมาก แต่ก็ไม่ใช่ว่ามีเพียงวิธีเดียวที่แก้ระบบพีชคณิตเชิงเส้นได้ เพราะ Secant Method ที่ดัดแปลงก็ใช้ในการแก้ระบบพีชคณิตไม่เชิงเส้นได้เหมือนกัน (อาจารย์ธราธรไม่เคยสอนตรง ๆ แต่พูดลอย ๆ ในห้อง ต้องไปจินตนาการต่อเอาเอง) สำหรับขั้นตอนนี้หากเรานิยาม $\underline{g} = \underline{y}^{i+1} - \underline{y}^i - h\underline{f}(\underline{y}^{i+1})$ จะเขียนในรูปการแปลงเชิงเส้นได้เป็น

$$\underline{\Delta y} = \underline{\underline{J}}^{-1}(-\underline{g})$$

สำหรับ $\underline{g}$ นั้นเราโปรแกรมให้คอมพิวเตอร์คำนวณได้ง่าย แต่สำหรับ Jacobian นั้นคงจะยากที่จะให้มันคำนวณอนุพันธ์โดยใช้สูตรตรง ๆ หากเราเขียน Jacobian ออกมาเราจะได้เป็น

$$\underline{\underline{J}} = \begin{pmatrix} | & | & | \\ \dfrac{\partial \underline{g}}{\partial y_1} & \dfrac{\partial \underline{g}}{\partial y_1} & \dfrac{\partial \underline{g}}{\partial y_1} \\ | & | & | \end{pmatrix}$$

ซึ่งเราสามารถประมาณค่าอนุพันธ์ย่อยเหล่านี้ได้โดยนิยามของมัน จะได้ว่า

$$\underline{\underline{J}} \approx \begin{pmatrix} | & | & | \\ \dfrac{\underline{g}(\underline{y} + \varepsilon[1\,0\,0]^\top) - \underline{g}(\underline{y})}{\varepsilon} & \dfrac{\underline{g}(\underline{y} + \varepsilon[0\,1\,0]^\top) - \underline{g}(\underline{y})}{\varepsilon} & \dfrac{\underline{g}(\underline{y} + \varepsilon[0\,0\,1]^\top) - \underline{g}(\underline{y})}{\varepsilon} \\ | & | & | \end{pmatrix} \quad (7)$$

เพียงเท่านี้เราก็สามารถโปรแกรมให้คอมช่วยเราแก้ระบบ IVP 3×3 ได้แล้ว ต่อมาเพื่อประมาณความคลาดเคลื่อนของตัวแปรต้นเรานิยามความสัมพันธ์ $\phi(T_{a0}) = T_{at}$ เมื่อ T_{a0} คืออุณหภูมิโดยรอบเริ่มต้น และ T_{at} คืออุณหภูมิโดยรอบที่ V = 10 (สังเกตว่าไม่มีตัวแปรที่เป็นเวกเตอร์แล้ว) หลังจากนี้เราก็จะวนกลับไปที่แนวคิดพื้นฐานเพื่อวนซ้ำจนได้คำตอบของ ODEs-BVP ออกมา โดยดูได้จากไฟล์ MATLAB code ที่เราแนบมาพร้อมกันได้เลย